

ME314 Homework 6

Submission instructions

Deliverables that should be included with your submission are shown in **bold** at the end of each problem statement and the corresponding supplemental material. **Your homework will be graded IFF you submit a single PDF, .mp4 videos of animations when requested and a link to a Google colab file that meet all the requirements outlined below.**

- List the names of students you've collaborated with on this homework assignment.
- Include all of your code (and handwritten solutions when applicable) used to complete the problems.
- Highlight your answers (i.e. **bold** and outline the answers) for handwritten or markdown questions and include simplified code outputs (e.g. `.simplify()`) for python questions.
- Enable Google Colab permission for viewing
- Click Share in the upper right corner
- Under "Get Link" click "Share with..." or "Change"
- Then make sure it says "Anyone with Link" and "Editor" under the dropdown menu
- Make sure all cells are run before submitting (i.e. check the permission by running your code in a private mode)
- Please don't make changes to your file after submitting, so we can grade it!
- Submit a link to your Google Colab file that has been run (before the submission deadline) and don't edit it afterwards!

NOTE: This Jupyter Notebook file serves as a template for you to start homework. Make sure you first copy this template to your own Google driver (click "File" -> "Save a copy in Drive"), and then start to edit it.

```
In [ ]: #Import cell
import sympy as sym
from sympy.abc import t
import numpy as np
import matplotlib.pyplot as plt
```

Problem 1 (20pts)

Show that if $R(\theta_1)$ and $R(\theta_2) \in SO(n)$ then the product is also a rotation matrix, that is $R(\theta_1)R(\theta_2) \in SO(n)$.

Hint 1: You know this is true when $n = 2$ by direct calculation in class, but for $n \neq 2$ you should use the definition of $SO(n)$ to verify it for arbitrary n . Do not try to do this by analyzing individual components of the matrix.

Turn in: A scanned (or photograph from your phone or webcam) copy of your handwritten solution. You can also use *LaTeX*. If you use SymPy, you need to include a copy of your code and

the code outputs. Make sure to note why your handwritten solution / code output explains the results.

i) Given $R(\theta_1)$ and $R(\theta_2) \in SO(n)$

$$\Rightarrow R(\theta_1)^T R(\theta_1) = I_{n \times n} \text{ and } \det(R(\theta_1)) = 1$$

Similarly

$$R(\theta_2)^T R(\theta_2) = I \text{ and } \det(R(\theta_2)) = 1$$

In order for the product $R(\theta_1) R(\theta_2)$ to be a rotation matrix we need to show

$$(i) \quad [R(\theta_1) R(\theta_2)]^T [R(\theta_1) R(\theta_2)] = I_{n \times n}$$

$$(ii) \quad \det(R(\theta_1) R(\theta_2)) = 1$$

$$\theta_1 \text{ and } \theta_2 \in \mathbb{R}$$

Proof (i) Consider the term $[R(\theta_1) R(\theta_2)]^T [R(\theta_1) R(\theta_2)]$

$$= (R(\theta_2)^T R(\theta_1)^T) (R(\theta_1) R(\theta_2))$$

$$\begin{aligned}
 & \text{Since } R(\theta_1) \in SO(n) \text{ and } R(\theta_2) \in SO(n) \\
 & = R(\theta_2)^T \left[R(\theta_1)^T R(\theta_1) \right] R(\theta_1) \\
 & \quad \downarrow \\
 & \quad I_{n \times n} \\
 & = R(\theta_2)^T R(\theta_2) \\
 & \quad \downarrow \\
 & \quad I_{n \times n} \\
 & = I_{n \times n}
 \end{aligned}$$

Proof (ii)

$$\begin{aligned}
 & \det(R(\theta_1) R(\theta_2)) \\
 & = \det(R(\theta_1)) \cdot \det(R(\theta_2)) \\
 & = 1 \times 1 \\
 & = 1
 \end{aligned}$$

$R(\theta_1) R(\theta_2)$ is also a rotation matrix i.e

$$R(\theta_1) R(\theta_2) \in SO(n)$$

Problem 2 (20pts)

Show that if $g(x_1, y_1, \theta_1)$ and $g(x_2, y_2, \theta_2) \in SE(2)$ then the product satisfies $g(x_1, y_1, \theta_1)g(x_2, y_2, \theta_2) \in SE(2)$.

Turn in: A scanned (or photograph from your phone or webcam) copy of your hand written solution. You can also use *LaTeX*. If you use *SymPy*, you need to include a copy of your code and the code outputs. Make sure to note why your handwritten solution / code output explains the results.

2) $g(x_1, y_1, \theta_1)$ and $g(x_2, y_2, \theta_2) \in SE(2)$ ← Given

$$g(x_1, y_1, \theta_1) = \begin{bmatrix} R(\theta_1) & \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} \\ 0 & 1 \end{bmatrix}_{3 \times 3} \quad g(x_2, y_2, \theta_2) = \begin{bmatrix} R(\theta_2) & \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} \\ 0 & 1 \end{bmatrix}_{3 \times 3}$$

where $R(\theta_1)$ and $R(\theta_2) \in SO(2)$ and

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} \in \mathbb{R}^2$$

To prove

$$g(x_1, y_1, \theta_1) \cdot g(x_2, y_2, \theta_2) \in SE(2)$$

$$= \begin{bmatrix} R(\theta_1) & \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} R(\theta_2) & \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} \\ 0 & 1 \end{bmatrix} \quad \begin{matrix} P_1 = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} \\ P_2 = \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} \end{matrix}$$

$$= \begin{bmatrix} \overset{2 \times 2}{R(\theta_1)R(\theta_2)} & \overset{2 \times 1}{R(\theta_1)P_2 + P_1} \\ \underset{1 \times 2}{0} & \underset{1 \times 1}{1} \end{bmatrix}_{3 \times 3} \quad \left[\text{which was proven in Problem 1} \right]$$

$R(\theta_1)R(\theta_2)$ is a rotation matrix i.e. $\in SO(2)$

$R(\theta_1)P_2 + P_1$ is a vector in the frame 1 and $\in \mathbb{R}^2$

∴ The above matrix is a homogeneous transformation matrix in $SE(2)$

Problem 3 (20pts)

Show that any homogeneous transformation in $SE(2)$ can be separated into a rotation and a translation. What's the order of the two operations, which comes first? What's different if we flip the order in which we compose the rotation and translation?

Hint 1: For the rotation and translation operation, we first need to know what's the reference frame for these two operations.

Turn in: A scanned (or photograph from your phone or webcam) copy of your hand written solution. You can also use *LaTeX*. If you use *SymPy*, you need to include a copy of your code and the code outputs. Make sure to note why your handwritten solution / code output explains the results.

5) let $g(x, y, \theta)$ be a homogeneous transformation in $SE(2)$

$$g(x, y, \theta) = \begin{bmatrix} R & \begin{bmatrix} x \\ y \end{bmatrix} \\ 0 & 1 \end{bmatrix}$$

We can write the above transformation matrix as below

$$= \begin{bmatrix} I & \begin{bmatrix} x \\ y \end{bmatrix} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R & \begin{bmatrix} 0 \\ 0 \end{bmatrix} \\ 0 & 1 \end{bmatrix}$$

Product of two $SE(2)$ matrices

You can use the result from Problem 2 and substitute
 $R(\theta_1) = I$ $R(\theta_2) = R$
 $P_1 = \begin{bmatrix} x \\ y \end{bmatrix}$ $P_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
 to get the decomposition

The first matrix is a pure translation operation and the second matrix is a pure rotation operation

$$g(x, y, \theta) = g(x, y, 0) \cdot g(0, 0, \theta)$$

$\uparrow$ $\uparrow$
 Pure translation Pure Rotation

The order of the multiplication cannot be change i.e we cannot write $g(x, y, \theta) = g(0, 0, \theta) \cdot g(x, y, 0)$ because homogenous transformations are not commutative

i.e $g = g_1 g_2 \neq g_2 g_1$

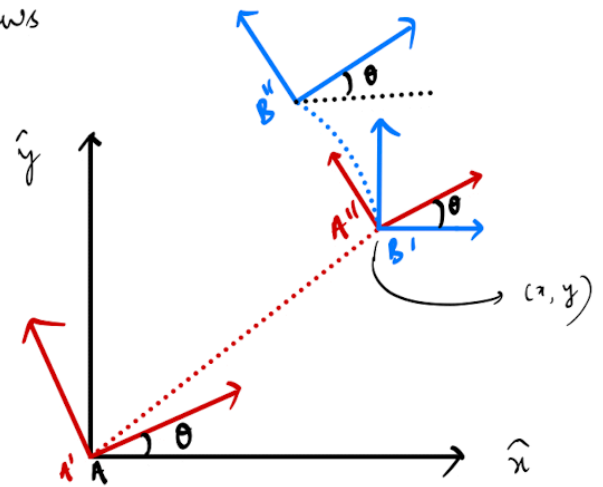
This can also be explained through the following diagram

$g(x, y, \theta)$ can be visualized as follows

First start with a axis aligned with A.

Then perform pure rotation θ about $\hat{z}$ axis of A. We get A'

Then perform pure translation of A' by $(x, y, 0)$ in A-axis to reach A'' .



If we reverse this operation i.e. if we start with an axis aligned with A, then perform a pure translation $(x, y, 0)$ in A-axis reach B' and then perform pure rotation θ about $\hat{z}$ axis of A we reach B'' . And $B'' \neq A''$

A'' will be given by $\begin{bmatrix} R & \begin{bmatrix} x \\ y \end{bmatrix} \\ 0 & 1 \end{bmatrix}$ where as

$$\begin{aligned} B'' & \text{ will be given by } g(0, 0, \theta) \cdot g(x, y, 0) \\ &= \begin{bmatrix} R & \begin{bmatrix} 0 \\ 0 \end{bmatrix} \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} I & \begin{bmatrix} x \\ y \end{bmatrix} \\ 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} R & R \begin{bmatrix} x \\ y \end{bmatrix} \\ 0 & 1 \end{bmatrix} \end{aligned}$$

Problem 4 (20pts)

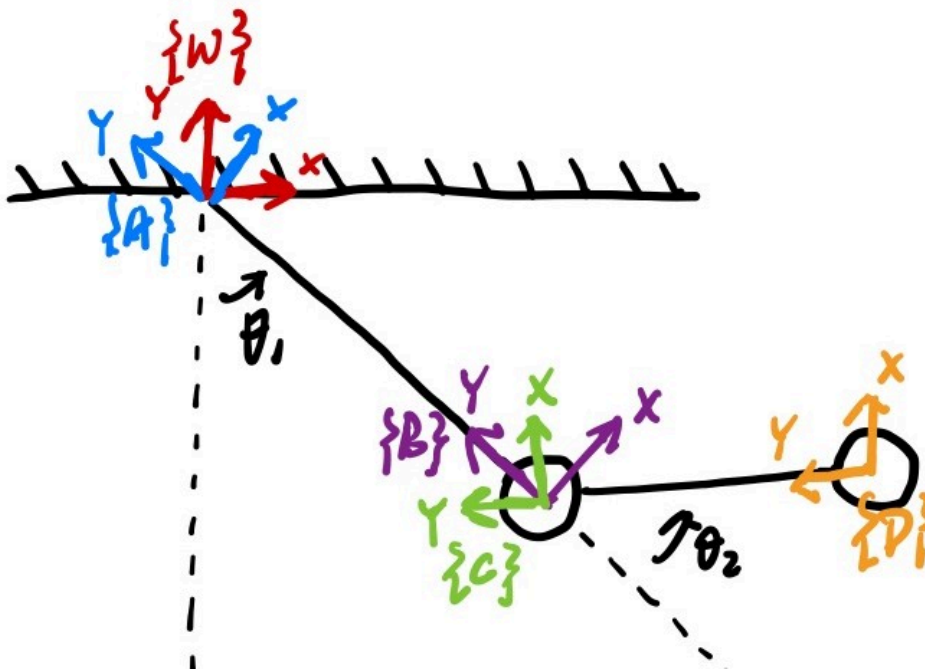
Simulate the same double-pendulum system in previous homework using only homogeneous transformation (and thus avoid using trigonometry). Simulate the system for $t \in [0, 3]$ with $dt = 0.01$. The parameters are $m_1 = m_2 = 1$, $R_1 = R_2 = 1$, $g = 9.8$ with initial conditions

$\theta_1 = \theta_2 = -\frac{\pi}{3}, \dot{\theta}_1 = \dot{\theta}_2 = 0$. Do not use functions provided in the modern robotics package for manipulating transformation matrices such as `RpToTrans()`, etc.

Hint 1: Same as in the lecture, you will need to define the frames by yourself in order to compute the Lagrangian. An example is shown below.

Turn in: Include a copy of your code used to simulate the system, and clearly labeled plot of θ_1 and θ_2 trajectory. Also, attach a figure showing how you defined the frames.

```
In [ ]: from IPython.core.display import HTML
display(HTML("<table><tr><td><img src='https://github.com/MuchenSun/ME314pngs/raw/master
```



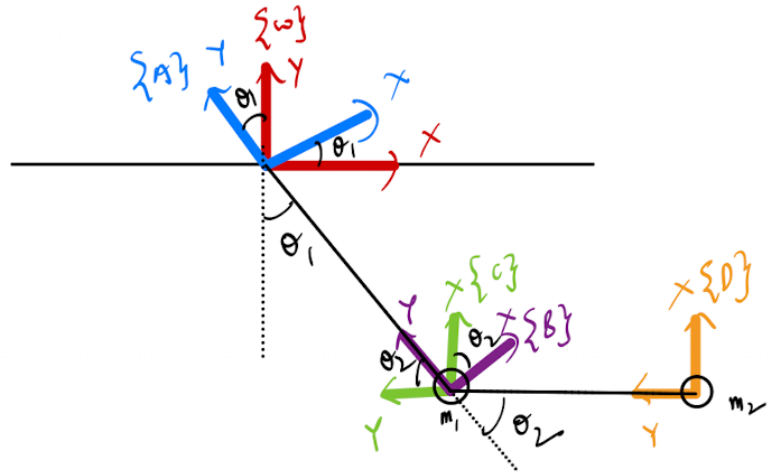
4)

$$g_{WA} = g(0, 0, \theta_1)$$

$$g_{AB} = g(0, -R_1, 0)$$

$$g_{BC} = g(0, 0, \theta_2)$$

$$g_{CO} = g(0, -R_2, 0)$$



$$g_{WA} = \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & 0 \\ \sin \theta_1 & \cos \theta_1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$g_{AB} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -R_1 \\ 0 & 0 & 1 \end{bmatrix}$$

$$g_{BC} = \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 & 0 \\ \sin \theta_2 & \cos \theta_2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$g_{CO} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -R_2 \\ 0 & 0 & 1 \end{bmatrix}$$

```
In [ ]: # define constants
R1, R2, m1, m2, g = sym.symbols('R1, R2, m1, m2, g')

# define the system state theta1 and theta2
theta1 = sym.Function(r'\theta_1')(t)
theta2 = sym.Function(r'\theta_2')(t)
# thetadot = theta.diff(t)
# thetaddot = thetadot.diff(t)

# We are interested in the position of the masses represented in {W} frame
# thus we need to the SE(2) transformation from {W} to {D}, and {W} to {B}

# Transformation from {W} to {A}, which is a rotation for theta1
g_wa = sym.Matrix([
    [sym.cos(theta1), -sym.sin(theta1), 0.],
    [sym.sin(theta1), sym.cos(theta1), 0.],
    [0., 0., 1.]
])
print('SE(2) transformation from {W} to {A}')
display(g_wa)

# {A} to {B} is just a translation along y-axis for -R1
g_ab = sym.Matrix([
    [1., 0., 0.],
    [0., 1., -R1],
    [0., 0., 1.]
])
1)
```

```

print('SE(2) transformation from {A} to {B}')
display(g_ab)

# Transformation from {B} to {C}, which is a rotation for theta2
g_bc = sym.Matrix([
    [sym.cos(theta2), -sym.sin(theta2), 0.],
    [sym.sin(theta2),  sym.cos(theta2), 0.],
    [0., 0., 1.]
])
print('SE(2) transformation from {B} to {C}')
display(g_wa)

# {C} to {D} is just a translation along y-axis for -R2
g_cd = sym.Matrix([
    [1., 0., 0.],
    [0., 1., -R2],
    [0., 0., 1.]
])
print('SE(2) transformation from {A} to {B}')
display(g_cd)

# Transformation from {W} to {B}
g_wb = g_wa * g_ab

print('SE(2) transformation from {W} to {B}')
display(g_wb)

# Transformation from {W} to {D}
g_wd = g_wa * g_ab * g_bc * g_cd

print('SE(2) transformation from {W} to {D}')
display(g_wd)

```

SE(2) transformation from {W} to {A}

$$\begin{bmatrix} \cos(\theta_1(t)) & -\sin(\theta_1(t)) & 0.0 \\ \sin(\theta_1(t)) & \cos(\theta_1(t)) & 0.0 \\ 0.0 & 0.0 & 1.0 \end{bmatrix}$$

SE(2) transformation from {A} to {B}

$$\begin{bmatrix} 1.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & -R_1 \\ 0.0 & 0.0 & 1.0 \end{bmatrix}$$

SE(2) transformation from {B} to {C}

$$\begin{bmatrix} \cos(\theta_1(t)) & -\sin(\theta_1(t)) & 0.0 \\ \sin(\theta_1(t)) & \cos(\theta_1(t)) & 0.0 \\ 0.0 & 0.0 & 1.0 \end{bmatrix}$$

SE(2) transformation from {A} to {B}

$$\begin{bmatrix} 1.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & -R_2 \\ 0.0 & 0.0 & 1.0 \end{bmatrix}$$

SE(2) transformation from {W} to {B}

$$\begin{bmatrix} 1.0 \cos(\theta_1(t)) & -1.0 \sin(\theta_1(t)) & R_1 \sin(\theta_1(t)) \\ 1.0 \sin(\theta_1(t)) & 1.0 \cos(\theta_1(t)) & -R_1 \cos(\theta_1(t)) \\ 0 & 0 & 1.0 \end{bmatrix}$$

SE(2) transformation from {W} to {D}

$$\begin{bmatrix} -1.0 \sin(\theta_1(t)) \sin(\theta_2(t)) + 1.0 \cos(\theta_1(t)) \cos(\theta_2(t)) & -1.0 \sin(\theta_1(t)) \cos(\theta_2(t)) - 1.0 \sin(\theta_2(t)) \cos(\theta_1(t)) & 1.0R_1 \\ 1.0 \sin(\theta_1(t)) \cos(\theta_2(t)) + 1.0 \sin(\theta_2(t)) \cos(\theta_1(t)) & -1.0 \sin(\theta_1(t)) \sin(\theta_2(t)) + 1.0 \cos(\theta_1(t)) \cos(\theta_2(t)) & -1.0R_1 \\ 0 & 0 & 0 \end{bmatrix}$$

```
In [ ]: # Position of mass m1 in {B}
x1_B, y1_B = 0., 0.
print('m1 position in {B}:')
display(sym.Matrix([x1_B, y1_B]))

# Homogeneous representation
X1_B = sym.Matrix([x1_B, y1_B, 1.])

# Position of mass m2 in {D}
x2_D, y2_D = 0., 0.
print('m2 position in {D}:')
display(sym.Matrix([x2_D, y2_D]))

# Homogeneous representation
X2_D = sym.Matrix([x2_D, y2_D, 1.])

# Position of m1 and m2 represented in W
X1_W = g_wb * X1_B
X2_W = g_wd * X2_D

x1_W = X1_W[0]
y1_W = X1_W[1]

x2_W = X2_W[0]
y2_W = X2_W[1]

print('m1 position in {W}:')
display(sym.Matrix([x1_W, y1_W]))

print('m2 position in {W}:')
display(sym.Matrix([x2_W, y2_W]))
```

m1 position in {B}:

$$\begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

m2 position in {D}:

$$\begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

m1 position in {W}:

$$\begin{bmatrix} 1.0R_1 \sin(\theta_1(t)) \\ -1.0R_1 \cos(\theta_1(t)) \end{bmatrix}$$

m2 position in {W}:

$$\begin{bmatrix} 1.0R_1 \sin(\theta_1(t)) - 1.0R_2 (-1.0 \sin(\theta_1(t)) \cos(\theta_2(t)) - 1.0 \sin(\theta_2(t)) \cos(\theta_1(t))) \\ -1.0R_1 \cos(\theta_1(t)) - 1.0R_2 (-1.0 \sin(\theta_1(t)) \sin(\theta_2(t)) + 1.0 \cos(\theta_1(t)) \cos(\theta_2(t))) \end{bmatrix}$$

```
In [ ]: x1_W_dot = x1_W.diff(t)
y1_W_dot = y1_W.diff(t)

x2_W_dot = x2_W.diff(t)
y2_W_dot = y2_W.diff(t)

KE1 = m1*(x1_W_dot**2 + y1_W_dot**2)/2
PE1 = m1*g*y1_W
```

```

KE2 = m2*(x2_W_dot**2 + y2_W_dot**2)/2
PE2 = m2*g*y2_W

KE = KE1 + KE2
PE = PE1 + PE2

lagrangian = KE - PE
lagrangian = sym.simplify(lagrangian)

# Display Lagrangian
print('\033[1mLagrangian of the system: ')
display(lagrangian)
print("\n")

# Wrap the lagrangian into a SymPy Matrix
L = sym.Matrix([lagrangian])
# display(L)

q = sym.Matrix([theta1, theta2])
qdot = q.diff(t)
qddot = qdot.diff(t)

# Compute Jacobian
dLdq = L.jacobian(q)
dLdqdot = L.jacobian(qdot)

print('\033[1mEuler Lagrangian Equations')
EL = dLdqdot.diff(t) - dLdq
# display(EL)

# LHS of the EL equations
lhs = sym.Matrix([exp for exp in EL])

# RHS of the EL equations
rhs = sym.Matrix([0 for exp in EL])

EL_eqns = sym.Eq(lhs, rhs)
# display(EL_eqns)

solns = sym.solve(EL_eqns, qddot, dict=True)

# for sol_ in solns:
#     print('\n\033[1mSolution: ')
#     for q_ in qddot:
#         display(sym.simplify(sym.Eq(q_, sol_[q_])))
#         print("\n")

# Substitute constants into symbolic solutions
subs_dict = {m1:1, m2:1, g:9.8, R1:1, R2:1}

c_sub_solns = [{q_: soln[q_].subs(subs_dict) for soln in solns for q_ in qddot}]

for c_sub_soln in c_sub_solns:
    print('\n\033[1mSolution: ')
    for q_ in qddot:
        display(sym.simplify(sym.Eq(q_, c_sub_soln[q_])))
        print("\n")

# Lamdifying the constant substituted solutions for theta1_ddot and theta2_ddot
state_vec = [theta1, qdot[0], theta2, qdot[1]]

```

```
lambdified_solns = [sym.lambdify(state_vec, c_sub_soln[q_]) for c_sub_soln in c_sub_soln
theta1ddot_func, theta2ddot_func = lambdified_solns
```

Lagrangian of the system:

$$0.5R_1^2m_1\left(\frac{d}{dt}\theta_1(t)\right)^2 + 1.0R_1gm_1\cos(\theta_1(t)) + 1.0gm_2(R_1\cos(\theta_1(t)) + R_2\cos(\theta_1(t) + \theta_2(t))) + 0.5m_2\left(R_1^2\left(\frac{d}{dt}\theta_1(t)\right)^2 + R_2^2\left(\frac{d}{dt}\theta_2(t)\right)^2 + 2R_1R_2\frac{d}{dt}\theta_1(t)\frac{d}{dt}\theta_2(t)\right) - 1.0R_2gm_2\cos(\theta_1(t) + \theta_2(t))$$

Euler Lagrangian Equations

Solution:

$$\frac{d^2}{dt^2}\theta_1(t) = \frac{-4.9\sin(\theta_1(t) + 2\theta_2(t)) + 14.7\sin(\theta_1(t)) - 1.0\sin(\theta_2(t))\left(\frac{d}{dt}\theta_1(t)\right)^2 - 2.0\sin(\theta_2(t))\frac{d}{dt}\theta_1(t)\frac{d}{dt}\theta_2(t) - 1.0\sin(\theta_2(t))\left(\frac{d}{dt}\theta_2(t)\right)^2}{\cos^2(\theta_2(t)) - 2}$$

$$-9.8\sin(\theta_1(t) - \theta_2(t)) + 9.8\sin(\theta_1(t) + \theta_2(t)) + 4.9\sin(\theta_1(t) + 2\theta_2(t)) - 14.7\sin(\theta_1(t)) + 3.0\sin(\theta_2(t))$$

$$\frac{d^2}{dt^2}\theta_2(t) = \frac{(2\theta_2(t))\frac{d}{dt}\theta_1(t)\frac{d}{dt}\theta_2(t) + 0.5\sin(2\theta_2(t))\left(\frac{d}{dt}\theta_2(t)\right)^2}{\cos^2(\theta_2(t)) - 2}$$

```
In [ ]: def integrate(f, xt, dt):
        """
        This function takes in an initial condition x(t) and a timestep dt,
        as well as a dynamical system f(x) that outputs a vector of the
        same dimension as x(t). It outputs a vector x(t+dt) at the future
        time step.

        Parameters
        =====
        dyn: Python function
            derivate of the system at a given step x(t),
            it can considered as \dot{x}(t) = func(x(t))
        xt: NumPy array
            current step x(t)
        dt:
            step size for integration

        Return
        =====
        new_xt:
            value of x(t+dt) integrated from x(t)
        """
        k1 = dt * f(xt)
        k2 = dt * f(xt+k1/2.)
        k3 = dt * f(xt+k2/2.)
        k4 = dt * f(xt+k3)
        new_xt = xt + (1/6.) * (k1+2.0*k2+2.0*k3+k4)
        return new_xt

def simulate(f, x0, tspan, dt, integrate):
    """
    This function takes in an initial condition x0, a timestep dt,
    a time span tspan consisting of a list [min_time, max_time],
    as well as a dynamical system f(x) that outputs a vector of the
    same dimension as x0. It outputs a full trajectory simulated
    over the time span of dimensions (xvec_size, time_vec_size).

    Parameters
```

```

=====
f: Python function
    derivate of the system at a given step x(t),
    it can considered as  $\dot{x}(t) = \text{func}(x(t))$ 
x0: NumPy array
    initial conditions
tspan: Python list
    tspan = [min_time, max_time], it defines the start and end
    time of simulation
dt:
    time step for numerical integration
integrate: Python function
    numerical integration method used in this simulation

Return
=====
x_traj:
    simulated trajectory of x(t) from t=0 to tf
"""
N = int((max(tspan)-min(tspan))/dt)
x = np.copy(x0)
tvec = np.linspace(min(tspan),max(tspan),N)
xtraj = np.zeros((len(x0),N))
for i in range(N):
    xtraj[:,i]=integrate(f,x,dt)
    x = np.copy(xtraj[:,i])
return xtraj

# Dynamics of the system
def double_pendulum(x):
    """
    System dynamics function (extended)

    Parameters
    =====
    x_dot: NumPy array
        s = [theta1, theta1_dot, theta2, theta2_ddot] is the extended system
        state vector

    Return
    =====
    x_ddot: NumPy array
        time derivative of input state vector,
        x_ddot = [theta1_dot, theta1_ddot, theta2_dot, theta2_ddot]
    """

    theta1_k, theta1_dot_k, theta2_k, theta2_dot_k = x

    theta1_ddot_k = theta1ddot_func(theta1_k, theta1_dot_k, theta2_k, theta2_dot_k)
    theta2_ddot_k = theta2ddot_func(theta1_k, theta1_dot_k, theta2_k, theta2_dot_k)

    return np.array([theta1_dot_k, theta1_ddot_k, theta2_dot_k, theta2_ddot_k])

```

```

In [ ]: # State vector is of the form (theta1, theta1_dot, theta2, theta2_ddot)
# Define the initial state
x0 = np.array([-np.pi/3, 0, -np.pi/3, 0])

# Simulation parameters
t0 = 0
tf = 3

```

```

dt = 0.01
N = int((tf - t0)/dt)
tvec = np.linspace(t0, tf, N)

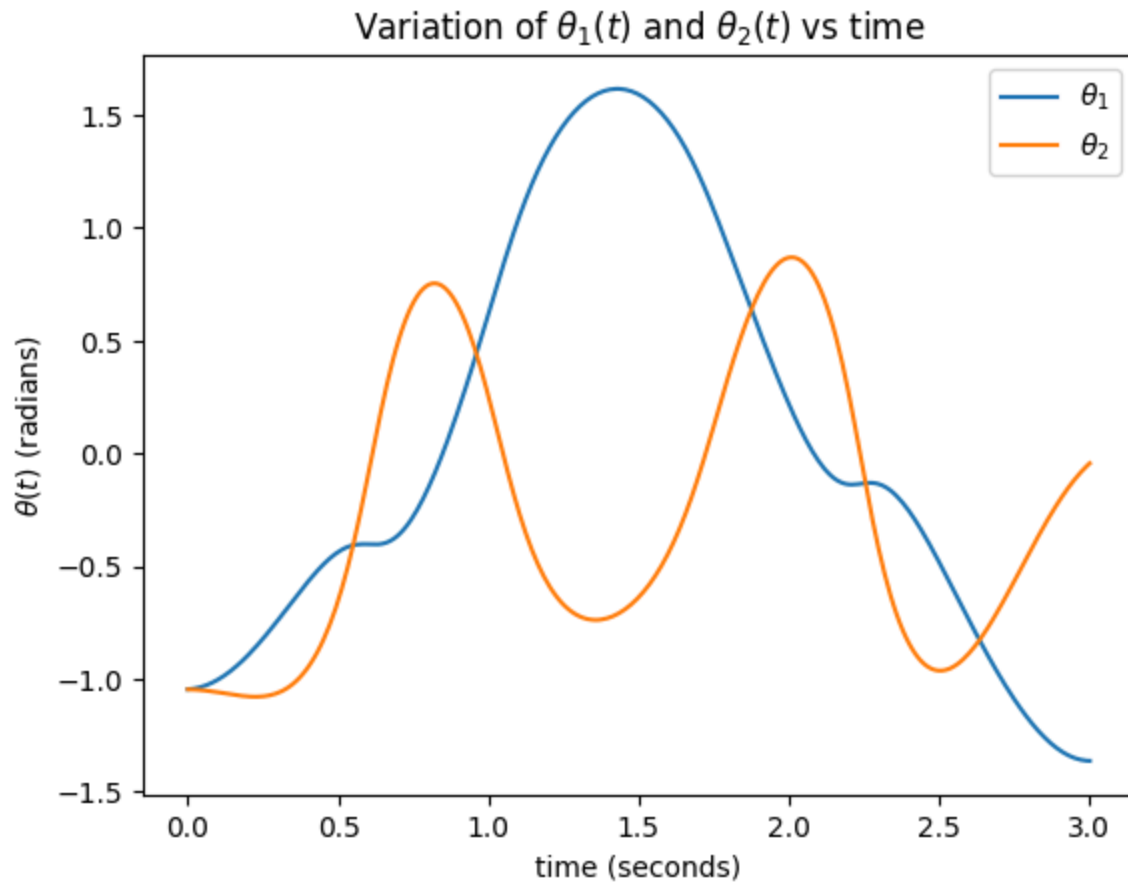
# print(double_pendulum(x0))
x_traj = simulate(double_pendulum, x0, [t0, tf], dt, integrate)

```

```

In [ ]: # plot
plt.plot(tvec, x_traj[0,:], label=r"$\theta_1$")
plt.plot(tvec, x_traj[2,:], label=r"$\theta_2$")
plt.title(r'Variation of $\theta_1(t)$ and $\theta_2(t)$ vs time')
plt.xlabel("time (seconds)")
plt.ylabel(r"$\theta(t)$ (radians)")
plt.legend()
plt.show()

```



Problem 5 (20pts)

Modify the previous animation function for the double-pendulum such that the animation shows the frames you defined in the last problem (it's similar to the `tf` in RViz, if you're familiar with ROS). All the x axes should be displayed in **green** and all the y axes should be displayed in **red**, with axis's length of 0.3 for all. An animation example can be found at <https://youtu.be/2H3KvRWQqys>. Do not use functions provided in the modern robotics package for manipulating transformation matrices such as `RpToTrans()`, etc.

Hint 1: Each axis can be considered as a line connecting the origin and the point $[0.3, 0]$ or $[0, 0.3]$ in that frame. You will need to use the homogeneous transformations to transfer these two axis/points back into the world/fixed frame. Example code showing how to display one frame is provided below.

Turn in: Include a copy of your code used for animation and a video of the animation. The video can be uploaded separately through Canvas, and it should be in ".mp4" format. You can either use screen capture or record the screen directly with your phone.

```
In [87]: def animate_double_pend(theta_array, L1=1, L2=1, T=10):
        """
        Function to generate web-based animation of double-pendulum system

        Parameters:
        =====
        theta_array:
            trajectory of theta1 and theta2, should be a NumPy array with
            shape of (2,N)
        L1:
            length of the first pendulum
        L2:
            length of the second pendulum
        T:
            length/seconds of animation duration

        Returns: None
        """

        #####
        # Imports required for animation.
        from plotly.offline import init_notebook_mode, iplot
        from IPython.display import display, HTML
        import plotly.graph_objects as go

        #####
        # Browser configuration.
        def configure_plotly_browser_state():
            import IPython
            display(IPython.core.display.HTML('''
                <script src="/static/components/requirejs/require.js"></script>
                <script>
                    requirejs.config({
                        paths: {
                            base: '/static/base',
                            plotly: 'https://cdn.plot.ly/plotly-1.5.1.min.js?noext',
                        },
                    });
                </script>
            '''))
        configure_plotly_browser_state()
        init_notebook_mode(connected=False)

        #####
        # Getting data from pendulum angle trajectories.
        xx1=L1*np.sin(theta_array[0])
        yy1=-L1*np.cos(theta_array[0])
        xx2=xx1+L2*np.sin(theta_array[0]+theta_array[1])
        yy2=yy1-L2*np.cos(theta_array[0]+theta_array[1])
        N = len(theta_array[0]) # Need this for specifying length of simulation

        #####
        # Define arrays containing data for frame axes
        # In each frame, the x and y axis are always fixed
        x_axis = np.array([0.3, 0.0])
        y_axis = np.array([0.0, 0.3])
```

```

# Use homogeneous transformation to transfer these two axes/points
# back to the fixed frame
frame_a_x_axis = np.zeros((2,N))
frame_a_y_axis = np.zeros((2,N))

origin_b_frame = np.zeros((2,N))
frame_b_x_axis = np.zeros((2,N))
frame_b_y_axis = np.zeros((2,N))

origin_c_frame = np.zeros((2,N))
frame_c_x_axis = np.zeros((2,N))
frame_c_y_axis = np.zeros((2,N))

origin_d_frame = np.zeros((2,N))
frame_d_x_axis = np.zeros((2,N))
frame_d_y_axis = np.zeros((2,N))

for i in range(N): # iteration through each time step
    # evaluate homogeneous transformation
    t_wa = np.array([[np.cos(theta_array[0][i]), -np.sin(theta_array[0][i]), 0],
                    [np.sin(theta_array[0][i]), np.cos(theta_array[0][i]), 0],
                    [0, 0, 1]])
    # transfer the x and y axes in fixed back to body frame at
    # the current time step
    frame_a_x_axis[:,i] = t_wa.dot([x_axis[0], x_axis[1], 1])[0:2]
    frame_a_y_axis[:,i] = t_wa.dot([y_axis[0], y_axis[1], 1])[0:2]

    t_ab = np.array([[1, 0, 0],
                    [0, 1, -L1],
                    [0, 0, 1]])

    t_wb = np.matmul(t_wa, t_ab)
    # Compute the origin of frame B in world coordinates
    origin_b_frame[:,i] = np.matmul(t_wb, np.array([0, 0, 1]))[0:2]

    frame_b_x_axis[:,i] = t_wb.dot([x_axis[0], x_axis[1], 1])[0:2] - origin_b_frame[:,i]
    frame_b_y_axis[:,i] = t_wb.dot([y_axis[0], y_axis[1], 1])[0:2] - origin_b_frame[:,i]

    t_bc = np.array([[np.cos(theta_array[1][i]), -np.sin(theta_array[1][i]), 0],
                    [np.sin(theta_array[1][i]), np.cos(theta_array[1][i]), 0],
                    [0, 0, 1]])

    t_wc = np.matmul(t_wb, t_bc)
    # Compute the origin of frame C in world coordinates
    origin_c_frame[:,i] = np.matmul(t_wc, np.array([0, 0, 1]))[0:2]

    frame_c_x_axis[:,i] = t_wc.dot([x_axis[0], x_axis[1], 1])[0:2] - origin_c_frame[:,i]
    frame_c_y_axis[:,i] = t_wc.dot([y_axis[0], y_axis[1], 1])[0:2] - origin_c_frame[:,i]

    t_cd = np.array([[1, 0, 0],
                    [0, 1, -L2],
                    [0, 0, 1]])

    t_wd = np.matmul(t_wc, t_cd)
    # Compute the origin of frame D in world coordinates
    origin_d_frame[:,i] = np.matmul(t_wd, np.array([0, 0, 1]))[0:2]

    frame_d_x_axis[:,i] = t_wd.dot([x_axis[0], x_axis[1], 1])[0:2] - origin_d_frame[:,i]
    frame_d_y_axis[:,i] = t_wd.dot([y_axis[0], y_axis[1], 1])[0:2] - origin_d_frame[:,i]

```

```
#####
# Using these to specify axis limits.
xm = -3 #np.min(xx1)-0.5
xM = 3 #np.max(xx1)+0.5
ym = -3 #np.min(yy1)-2.5
yM = 3 #np.max(yy1)+1.5

#####
# Defining data dictionary.
# Trajectories are here.
data=[
    # note that except for the trajectory (which you don't need this time),
    # you don't need to define entries other than "name". The items defined
    # in this list will be related to the items defined in the "frames" list
    # later in the same order. Therefore, these entries can be considered as
    # labels for the components in each animation frame
    dict(name='Arm'),
    dict(name='Mass 1'),
    dict(name='Mass 2'),
    dict(name='World Frame X'),
    dict(name='World Frame Y'),
    dict(name='A Frame X Axis'),
    dict(name='A Frame Y Axis'),
    dict(name='B Frame X Axis'),
    dict(name='B Frame Y Axis'),
    dict(name='C Frame X Axis'),
    dict(name='C Frame Y Axis'),
    dict(name='D Frame X Axis'),
    dict(name='D Frame Y Axis'),

    # You don't need to show trajectory this time,
    # but if you want to show the whole trajectory in the animation (like what
    # you did in previous homeworks), you will need to define entries other than
    # "name", such as "x", "y". and "mode".

    # dict(x=xx1, y=yy1,
    #     mode='markers', name='Pendulum 1 Traj',
    #     marker=dict(color="fuchsia", size=2)
    # ),
    # dict(x=xx2, y=yy2,
    #     mode='markers', name='Pendulum 2 Traj',
    #     marker=dict(color="purple", size=2)
    # ),
    ]

#####
# Preparing simulation layout.
# Title and axis ranges are here.
layout=dict(autosize=False, width=1000, height=1000,
    xaxis=dict(range=[xm, xM], autorange=False, zeroline=False,dtick=1),
    yaxis=dict(range=[ym, yM], autorange=False, zeroline=False,scaleanchor =
    title='Double Pendulum Simulation',
    hovermode='closest',
    updatemenus= [{ 'type': 'buttons',
        'buttons': [{ 'label': 'Play', 'method': 'animate',
            'args': [None, { 'frame': { 'duration': T, 're
        { 'args': [[None], { 'frame': { 'duration': T, '
            'transition': { 'duration': 0} }], 'label': 'Pa
    ]
}]
```

```

#####
# Defining the frames of the simulation.
# This is what draws the lines from
# joint to joint of the pendulum.
frames=[dict(data=[# first three objects correspond to the arms and two masses,
# same order as in the "data" variable defined above (thus
# they will be labeled in the same order)
dict(x=[0,xx1[k],xx2[k]],
y=[0,yy1[k],yy2[k]],
mode='lines',
line=dict(color='orange', width=3),
),
go.Scatter(
x=[xx1[k]],
y=[yy1[k]],
mode="markers",
marker=dict(color="blue", size=12)),
go.Scatter(
x=[xx2[k]],
y=[yy2[k]],
mode="markers",
marker=dict(color="blue", size=12)),
# display x and y axes of the fixed frame in each animation frame
dict(x=[0,x_axis[0]],
y=[0,x_axis[1]],
mode='lines',
line=dict(color='green', width=3),
),
dict(x=[0,y_axis[0]],
y=[0,y_axis[1]],
mode='lines',
line=dict(color='red', width=3),
),
# display x and y axes of the {A} frame in each animation frame
dict(x=[0, frame_a_x_axis[0][k]],
y=[0, frame_a_x_axis[1][k]],
mode='lines',
line=dict(color='green', width=3),
),
dict(x=[0, frame_a_y_axis[0][k]],
y=[0, frame_a_y_axis[1][k]],
mode='lines',
line=dict(color='red', width=3),
),
# display x and y axes of the {B} frame in each animation frame
dict(x=[origin_b_frame[0][k], origin_b_frame[0][k] + frame_b_x_ax
y=[origin_b_frame[1][k], origin_b_frame[1][k] + frame_b_x_ax
mode='lines',
line=dict(color='green', width=3),
),
dict(x=[origin_b_frame[0][k], origin_b_frame[0][k] + frame_b_y_ax
y=[origin_b_frame[1][k], origin_b_frame[1][k] + frame_b_y_ax
mode='lines',
line=dict(color='red', width=3),
),
# display x and y axes of the {C} frame in each animation frame
dict(x=[origin_c_frame[0][k], origin_c_frame[0][k] + frame_c_x_ax
y=[origin_c_frame[1][k], origin_c_frame[1][k] + frame_c_x_ax
mode='lines',
line=dict(color='green', width=3),

```

```

    ),
    dict(x=[origin_c_frame[0][k], origin_c_frame[0][k] + frame_c_y_ax
          y=[origin_c_frame[1][k], origin_c_frame[1][k] + frame_c_y_ax
          mode='lines',
          line=dict(color='red', width=3),
          ),
    # display x and y axes of the {D} frame in each animation frame
    dict(x=[origin_d_frame[0][k], origin_d_frame[0][k] + frame_d_x_ax
          y=[origin_d_frame[1][k], origin_d_frame[1][k] + frame_d_x_ax
          mode='lines',
          line=dict(color='green', width=3),
          ),
    dict(x=[origin_d_frame[0][k], origin_d_frame[0][k] + frame_d_y_ax
          y=[origin_d_frame[1][k], origin_d_frame[1][k] + frame_d_y_ax
          mode='lines',
          line=dict(color='red', width=3),
          ),

    ]) for k in range(N)]

#####
# Putting it all together and plotting.
figure1=dict(data=data, layout=layout, frames=frames)
iplot(figure1)

```

In [88]: `animate_double_pend(np.delete(x_traj, [1, 3], axis=0), L1=1, L2=1, T=tf-t0)`

Double Pendulum Simulation

Play

Pause

